Thread Orchestrator (Bachi)

Shio Ramen Snake-Traversal & Work Distribution Pipeline

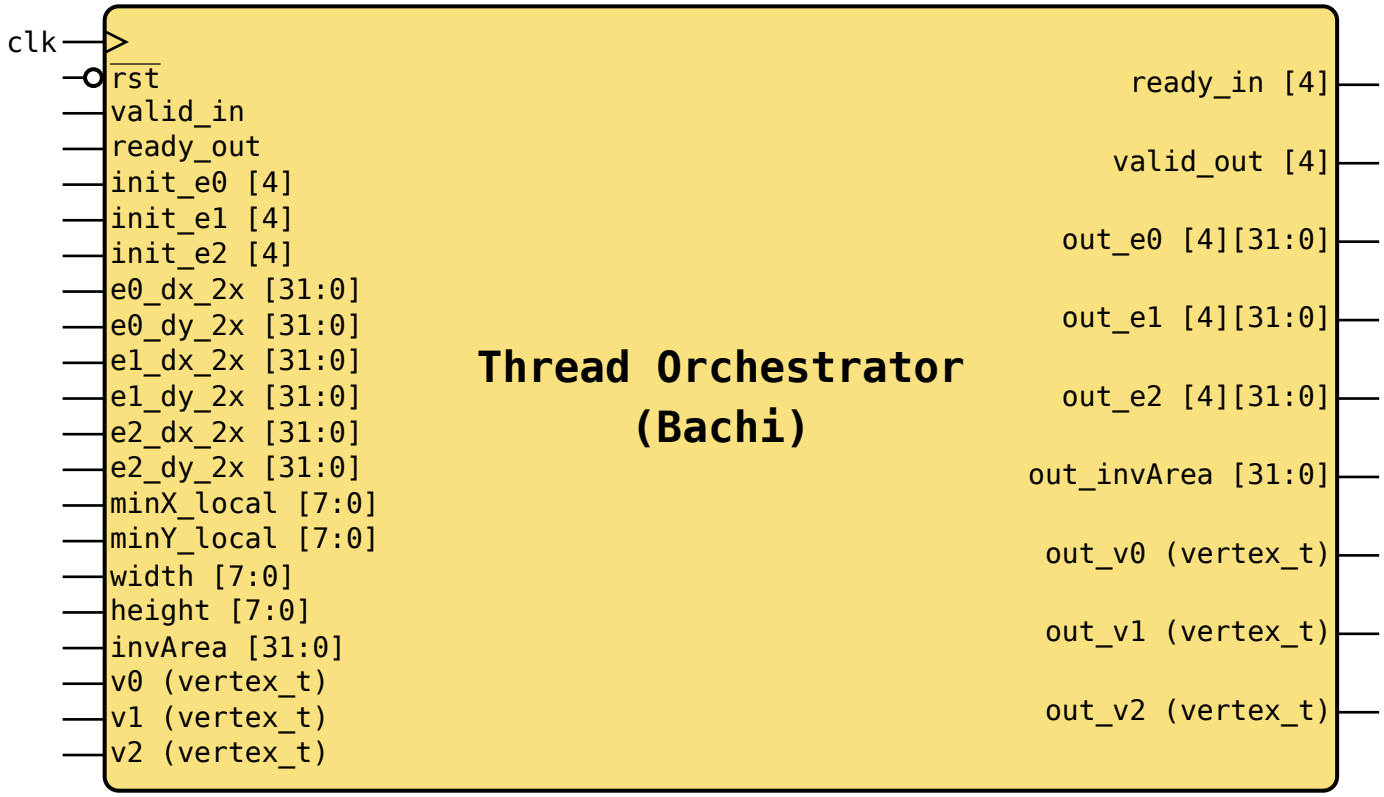
1. Module Overview

The `thread_orchestrator` module (codename **Bachi**, named after the sweeping plectrum used for traditional string instruments) is responsible for traversing the 2D bounding box of a triangle inside a specific tile. It performs a snake-like (boustrophedon) traversal, evaluating 2×2 pixel quads in each step, and dynamically updates the edge functions for all 4 pixels of the quad in parallel.

Rather than re-evaluating the full edge function at each step, Bachi performs incremental floating-point additions/subtractions based on pre-calculated derivatives ($e_{i,\,\mathrm{dx}}$ and $e_{i,\,\mathrm{dy}}$), significantly reducing hardware overhead. The updated edge functions and metadata are then distributed to four parallel downstream thread execution units.

2. Block Diagram

The block diagram below details Bachi's input and output ports:



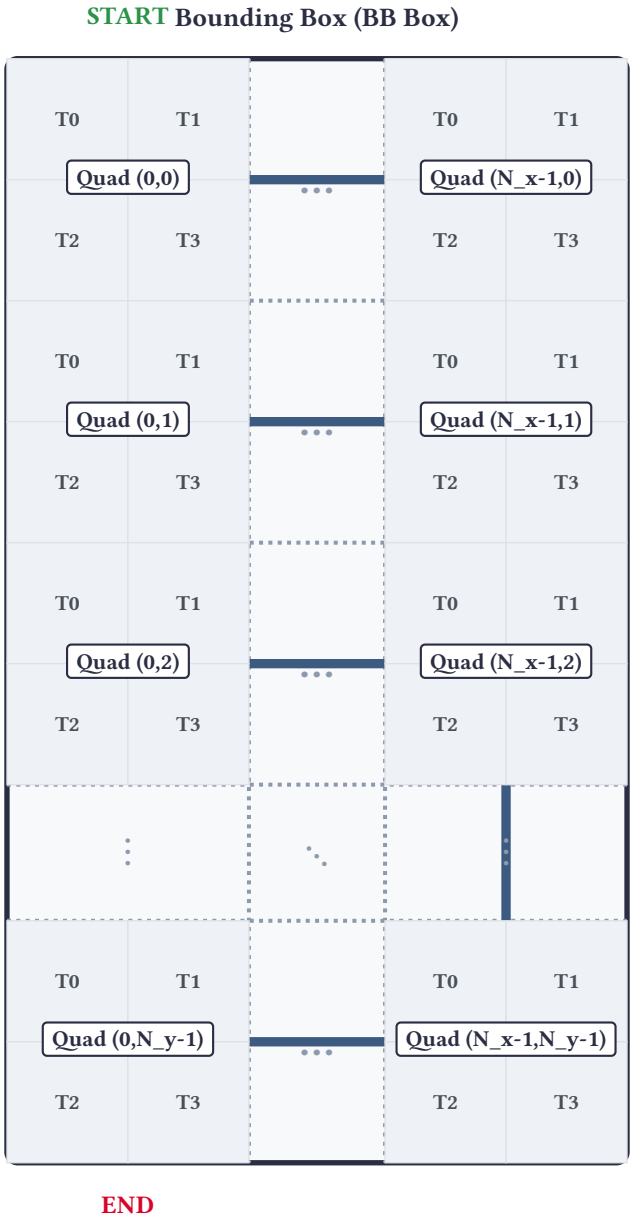
2. Module Interface

The table below specifies the functionality and description of each port on Bachi:

Port Name	Direction	Type / Width	Description
clk	Input	logic	System Clock (100 MHz)
rst_n	Input	logic	System Reset (Active Low)
valid_in	Input	logic	Handshake valid from Frontend
ready_out	Output	logic	Handshake ready to Frontend (High in ST_IDLE)
ready_in [4]	Input	logic	Handshake ready from 4 thread units
valid_out [4]	Output	logic	Handshake valid to 4 thread units
init_e0 [4], init_e1 [4], init_e2 [4]	Input	logic [31:0]	Initial edge functions evaluated at top-left pixel quad coordinates
e0_dx_2x, e0_dy_2x	Input	logic [31:0]	Edge function 0 derivatives scaled by 2 (float)
e1_dx_2x, e1_dy_2x	Input	logic [31:0]	Edge function 1 derivatives scaled by 2 (float)
e2_dx_2x, e2_dy_2x	Input	logic [31:0]	Edge function 2 derivatives scaled by 2 (float)
minX_local, minY_local	Input	logic [7:0]	Starting pixel coordinates of local bounding box
width, height	Input	logic [7:0]	Width and height of the bounding box (pixels)
invArea	Input	logic [31:0]	Inverse triangle area (floating-point recip)
v0, v1, v2	Input	vertex_t	Vertex coordinate descriptors
out_e0 [4], out_e1 [4], out_e2 [4]	Output	logic [31:0]	Traversally updated edge functions (1 per thread)
out_invArea	Output	logic [31:0]	Pipelined inverse area (float)
out_v0, out_v1, out_v2	Output	vertex_t	Pipelined vertex coordinates

4. Boustrophedon Traversal (Snake Path)

Bachi performs a boustrophedon (snake-like) traversal of 2×2 pixel quads to sweep the bounding box of the triangle efficiently. The diagram below illustrates the traversal path across a grid of quads (each representing a 2×2 pixel block):



The FSM tracks the current horizontal quad index `curr_qx` ($\in [0, \text{num_quads_x} - 1]$) and vertical quad index `curr_qy` ($\in [0, \text{num_quads_y} - 1]$), where:

$$\text{num_quads_x} = \text{width} \gg 1$$

$$\text{num_quads_y} = \text{height} \gg 1$$

The sweep direction is controlled by the register `dir_right`:

- **Rightward Sweep (`dir_right = 1`)**: Moves horizontally to the right ($\Delta x = +2$ pixels). At the right boundary, it moves down ($\Delta y = +2$ pixels) and flips `dir_right` to 0.
- **Leftward Sweep (`dir_right = 0`)**: Moves horizontally to the left ($\Delta x = -2$ pixels). At the left boundary, it moves down ($\Delta y = +2$ pixels) and flips `dir_right` to 1.
- **Termination**: Upon reaching the end of the final row boundary, traversal terminates, asserting `step_done` and returning to `ST_IDLE`.